

Utilisation de VS Code

- Présenter l'outils
- Présenter les différentes fonctionnalités (les extensions, partie Git)
- Set up de base (raccourci, etc)

Explication des outils de Debug (navigateur & console)

- Petite présentation des outils de debug
- La console, comment ça fonctionne, comment afficher les éléments, le console.log
- Debugger, et la limite du console.log()
- Turbo console log
- Sources, et point d'arrêt

Création et structuration d'une page HTML avec du CSS

- Créer une page web avec uniquement HTML & CSS
- Cette page doit représenter dans un premier espace, une todo list avec les champs suivants :
 - o Le titre (input)
 - o La description (texture)
 - o Une case à cocher qui représente si la tâche a été faites
 - o L'Id de la tâche (un nombre)
 - o Un bouton « Ajouter une tâche » avec une icône « + » pour soumettre le formulaire
- Au niveau de la structure, ces champs doivent être représenter sous la forme d'un formulaire
- Ensuite il faut afficher une liste de tâches
- Chaque tâche doit être affiché sous la forme d'une « carte » qui contient toutes ses informations (voir figure 1)

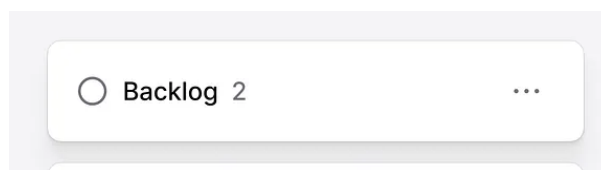


Figure 1 : Exemple d'affichage d'une tâche

- Si la tâche est faite, la case doit être cochée, le fond de la carte doit avoir une autre couleur, et le titre doit être barré. Cet état doit être représenté par une classe CSS qui viendra s'ajouter aux cartes faites
- Pour le début, faites une page avec 5 tâches qui doivent être faites, ou qui le sont déjà

Formulaire avec affichage d'une réponse dynamique (en javascript)

Ici, nous allons ajouter toute la partie dynamique à notre todo list.

Dans un premier temps, au lieu d'afficher un nombre d'éléments statique (5 tâches), il faut créer un tableau d'objet de ce type (voir figure 2).



Figure 2 : Type d'objet utilisé dans la todo list

Le but premier est de créer une tâche et l'ajouter dans ce tableau lorsque l'on remplit le formulaire et le soumet via le bouton.

Ensuite, une fois notre formulaire fonctionnel, il faut afficher dynamiquement notre tableau dès qu'il a été mis à jour (par exemple lorsque l'on ajoute un élément).

Pour cela, on va créer une fonction « render() » qui s'occupera d'afficher dynamiquement notre liste de todo, ainsi on centralise l'affichage et évite les problèmes par la suite.

Enfin, on va ajouter une icône d'une poubelle sur chacune de nos tâches et créer toute la chaîne qui nous permettra, lors du clic sur cette icône, de supprimer notre tâche (capture de l'événement, suppression de la tâche dans le tableau, réaffichage du tableau).

Appels fetch & GET vers une API existante

Cette partie nécessite l'utilisation en local du back reçu à la fin du TD1

Maintenant que nous avons une todo liste dynamique côté front, on va commencer à mettre en place la communication avec le BE afin de stocker nos données de façon continue.

Premièrement, nous allons créer un service qui va s'occuper de faire la communication avec l'API afin de centraliser cette logique.

Ensuite, nous allons prendre connaissance de la documentation API avant de créer nos différentes fonctions pour récupérer les données.

Enfin, grâce à l'API fetch de Javascript, nous allons créer nos différentes requêtes HTTP afin de récupérer et envoyer nos données à l'API.

Une fois notre service mis en place, testé et fonctionnel, nous allons pouvoir l'utiliser dans notre todo list et lié directement nos données à celles reçu par le backend.